

ARCHITECTURE DECISION RECORD

Engineering Decisions, *Explained.*

How the company brain stays **fast, correct, and safe**: deduplicating enterprise data by identity rather than content, enforcing permissions twice, and routing every query through a tiered fleet of language models — each step assigned the smallest model that can do the job well.

1× COPY STORED PER EMAIL, REGARDLESS OF RECIPIENTS	2-stage ACL ENFORCEMENT — PRE-FILTER + LIVE RE-CHECK	3-tier MODEL STRATEGY: ACK · PLAN · THINK	44s→6s QUERY LATENCY AFTER THE ACL_RECHECK POST-MORTEM
--	--	---	--

IN THIS DOCUMENT

01	Duplicate Data Handling	Source-native identity, ACL union, idempotent index writes	p.2
02	ACL Enforcement	Pre-filter for speed, re-check for truth — and the 44-second war story	p.3
03	LLM Orchestration	The right model for each job; thinking as a measured dial	p.4
04	Systemic Choices	Hybrid retrieval, graceful degradation, one identity model	p.4

STACK

FastAPI · Azure AI Search · Cosmos Gremlin · ADX
Azure Container Apps — Central India

SOURCES INDEXED

Outlook Mail · Calendar · Teams · SharePoint · Slack
generated from substrateos-api/ — every claim code-verified

Duplicate data is an *identity* problem, not a hashing problem.

An email sent to five people exists five times in Microsoft Graph — once per mailbox. A calendar invite exists once per attendee. Naïve ingestion would embed, index, and store each copy, multiplying cost and polluting retrieval with near-identical hits.

THE DECISION

Deduplicate on each source's **native globally-unique identifier** — not content hashes. Identity is stable across re-syncs, mailboxes, and replicas, so re-ingestion is naturally idempotent, with zero hash-collision edge cases and zero "is this the same document?" heuristics.

SOURCE	DEDUP KEY	DOCUMENT ID	EFFECT
Outlook Mail	<code>internetMessageId</code>	<code>outlookmail:{imid}</code>	Same email across N mailboxes collapses to one document
Outlook Calendar	<code>iCalUID</code>	<code>outlookcal:{ical}</code>	One event, shared by organizer and every attendee
Teams	message ID	<code>teams:{team}:{ch}:{msg}</code>	Structurally unique composite key
SharePoint	item ID	<code>sp:{site}:{item_id}</code>	SharePoint's native per-site identifier

→ ACL union on collision — never duplication

When the same message surfaces in five mailboxes, `_merge_into()` in `connectors/outlook_mail.py` keeps **one** document and **unions** its `acl_principals`. Every legitimate recipient can still retrieve it, but the index, the embedding bill, and the answer context see exactly one copy. **Storage scales with unique content, not recipient count.**

→ Deterministic chunk IDs → idempotent index writes

Chunks are keyed `{doc_id}#{chunk}-{index}` and written with Azure AI Search's `merge_or_upload_documents`. A re-sync **overwrites in place** rather than accumulating stale duplicates — there is no "purge, then reload" window during which search returns nothing.

→ Graph delta tokens for incremental sync

`@odata.deltaLink` tokens are persisted per user and resource (`connectors/subscriptions.py`); each sweep fetches **only what changed** since the last run. The token doubles as an idempotency checkpoint: a crashed sweep simply resumes from the last committed delta, and Graph API cost stays bounded no matter how large the tenant grows.



Enforce permissions twice: *pre-filter* for speed, *re-check* for truth.

A knowledge system that answers from other people's email lives or dies on access control. One check is never enough — index-time ACLs go stale the moment someone is removed from a thread.

THE DECISION

Stage 1 pushes the user's principals into the search engine itself as an OData filter, so unauthorized chunks never leave Azure AI Search. **Stage 2** re-validates every retrieved candidate against a live Redis ACL store before generation — revocations take effect **immediately**, without waiting for re-indexing. And the re-check **fails closed**: if Redis is unreachable, candidates are dropped, never leaked.

- **Pre-filter, inside the engine:** `build_acl_filter(user)` composes `tenant_id eq '...'` and `acl_principals/any(p: search.in(p, ...))` — tenant scoping and principal membership enforced by Azure AI Search, not by app code that could forget to call it.
- **Live re-check before answering:** `ACLStore.recheck()` validates each candidate against per-document principal sets (`acl:doc:{tenant}:{doc_id}`), with a configurable key-miss fallback to index-time ACLs for single-tenant pilots.
- **Per-participant ACLs at ingest:** every email and event is stamped with its owner's Entra object ID; combined with the dedup union (§01), one stored document is visible to exactly the people who legitimately hold a copy.
- **Identity, not shared secrets, for admin:** the admin panel is gated by Entra "Admin" group membership — token claims first, app-only Graph lookup (10-min cache) as fallback, fail-closed on Graph errors. The browser admin key was deleted outright.
- **Machine access inherits human limits:** Context-API and MCP requests resolve `sbx_*` personal access tokens (SHA-256-hashed in Cosmos) to a real user, then run the **identical** ACL-scoped pipeline. An agent can never see more than the human behind its token.

POST-MORTEM

The 44-second query

SYMPTOM

Production queries in the India deployment took ~44 s and returned **empty answers** — every candidate silently dropped.

ROOT CAUSE

The ACL store built a Redis client even with no Redis host configured. Each re-check hit a dead socket with **no timeout**: $\sim 7\text{ s} \times 6\text{ docs} \approx 44\text{ s}$, then fail-closed discarded everything.

FIX • COMMIT 7530561

Explicit **no-op mode** when unconfigured, plus 2-second socket timeouts as defense in depth. Warm-path latency returned to ~6 s.

*Every external dependency needs a deliberate “absent” mode and a bounded timeout — **fail-closed must never mean hang-closed.***



No single model answers a query.

Each pipeline step gets the smallest model that can do it well — and the thinking budget is a measured dial, not a default. Profiling showed ~1,300 unconstrained thinking tokens cost ≈ 13 s; the synthesis cap of 256 keeps the reasoning benefit while bounding tail latency.

TIER 1 · REFLEX< 1 s

flash-lite

thinking 0 · 60 tok · t=0.5

Instant, context-aware “On it...” acknowledgement while the real answer is computed. Deterministic template as fallback.

TIER 2 · ROUTINGfast

flash

thinking 0 · 60-200 tok · t=0.0

Rewrites the query, extracts entities, decides live-fetch, and routes to skills via strict JSON. Degrades to a keyword heuristic on failure.

TIER 3 · SYNTHESIS ≈ 6 s warm

pro · thinking

thinking ≤ 256 · 800 tok · t=0.0

Grounded, citation-enforced answer over ranked candidates — the only step that earns a thinking budget.

- Every LLM step has a non-LLM escape hatch — planner → freshness heuristic, ack → template, explicit /slug skips routing. An LLM outage degrades quality, never availability.
- Caching can't cross permission lines: answers cache for 600 s keyed on {tenant, principals, query} — the principal set in the key makes cross-user leakage structurally impossible. Embeddings cache 24 h by SHA-256.

Choices that hold the system together.

a. Hybrid retrieval, fused ranking

One query runs vector (3072-dim embeddings) + BM25 + semantic rerank, merged by Reciprocal Rank Fusion (K=60), then re-ranked with people-proximity (Cosmos graph) and recency-weighted engagement signals (ADX). No single signal is trusted alone.

b. Graceful degradation as policy

Redis, Cosmos, and ADX are all optional at runtime: cache → no-op, history → empty, activity score → 0, live-fetch → 600 ms hard timeout. Each failure narrows the answer; none breaks the answer path.

c. One identity model, four doors

resolve_user() orders Easy Auth header → PAT bearer → Entra JWT → debug override. The web SPA, MCP server, Context API, and scripts all converge on one User object — and one ACL evaluation.

d. MCP on the production stack

FastMCP mounts at /mcp (streamable HTTP, stateless); a ContextVar threads the PAT-resolved user into tools. Agents reuse the orchestrator verbatim — no second, weaker security path.

e. Citations as a contract

The synthesis prompt requires bracketed markers; parse_citations_from_answer() maps them to source metadata and strips orphans. Every claim a user reads traces to a source they may see.

f. Idempotent background sync

A lifespan-owned scheduler runs directory sync and subscription renewals as idempotent ticks; one tick's exception is swallowed and retried next interval — no dead loops, no duplicate state.